

Kth Node From the End of a Linked List

Java Code + Visualization

1. Java Code

```
class Solution {
    public int getKthFromLast(Node head, int k) {

        int len = length(head);

        if (k > len) {
            return -1;
        }

        int steps = len - k;

        while (steps > 0) {
            head = head.next;
            steps--;
        }

        return head.data;
    }

    public static int length(Node head) {

        int count = 0;

        while (head != null) {
            count++;
            head = head.next;
        }

        return count;
    }
}
```

2. Visualization

Example:

10 → 20 → 30 → 40 → 50 → null

k = 2

2nd node from the end:

10 → 20 → 30 → 40 → 50
 ↑
 2nd

Answer = 40

3. Find Length

10 → 20 → 30 → 40 → 50 → null

10 → count = 1

20 → count = 2

30 → count = 3

40 → count = 4

50 → count = 5

len = 5

4. Calculate Steps

```
k = 2
len = 5
```

```
steps = len - k
       = 5 - 2
       = 3
```

5. Move Head

Initial:

```
10 → 20 → 30 → 40 → 50
  ↑
head
```

Move 1:

```
10 → 20 → 30 → 40 → 50
    ↑
   head
```

Move 2:

```
10 → 20 → 30 → 40 → 50
          ↑
        head
```

Move 3:

```
10 → 20 → 30 → 40 → 50
                ↑
              head
```

```
steps = 0
head.data = 40
```

6. Another Example

```
1 → 2 → 3 → 4 → 5 → 6 → null
```

```
k = 4
len = 6
```

```
steps = 6 - 4 = 2
```

Move 1:

```
1 → 2 → 3 → 4 → 5 → 6
  ↑
```

Move 2:

```
1 → 2 → 3 → 4 → 5 → 6
      ↑
```

Answer = 3

7. If k > length

```
10 → 20 → 30 → null
```

```
len = 3
k = 5
```

```
k > len
↓
true
↓
return -1
```

Output = -1

8. Complete Flow

```
Linked List
↓
Find Length
↓
k > length?
/      \
Yes      No
↓        ↓
-1      steps = len - k
        ↓
        Move head
        ↓
        steps = 0
        ↓
        head.data
```

9. Complexity

Time → $O(n)$
Space → $O(1)$